

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №2
Проектирование и технический дизайн API

Выполнил:
Христофоров Владислав

Группа
К3340

Поток
БР 1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Анализ и формирование требований к API

Функциональные требования: Программный интерфейс (API) должен обеспечивать реализацию следующей бизнес-логики:

- Управление пользователями (IAM): регистрация пользователей, аутентификация по email и паролю с выдачей токена, просмотр и редактирование личного профиля (включая обновление URL-адреса аватара `avatar_url` и биографии).
- Разграничение доступа (RBAC): защита эндпоинтов изменения данных. Гости могут только читать контент. Авторизованные пользователи могут управлять только своими записями, но могут взаимодействовать с чужими (лайки, комменты).
- Управление контентом: выполнение полного цикла CRUD (создание, чтение, обновление, удаление) для текстовых постов блога и рецептов. При создании рецепта бэкенд должен обрабатывать сохранение вложенных данных: шагов приготовления (`RecipeStep`) и связей с ингредиентами (`RecipeIngredient`).
- Поиск и фильтрация: получение коллекций рецептов и постов с реализацией пагинации (`limit/offset`) и фильтрации (например, фильтрация рецептов по сложности `difficulty` или категории `dish_type_id`).

- Социальные взаимодействия: реализация бизнес-логики подписок на авторов (Subscription), комментирования контента, сохранения рецептов в личную коллекцию (закладки) и системы лайков.

- Валидация входящих данных: проверка корректности данных на стороне бэкенда до обращения к БД (проверка формата email, проверка того, что `cooking_time_minutes` и `amount` ингредиентов строго больше нуля).

Нефункциональные требования:

- Формат данных и архитектура: API должно соответствовать принципам REST, обмен данными между клиентом и сервером осуществляется строго в формате `application/json`.

- Безопасность аутентификации: использование механизма Stateless сессий с помощью JSON Web Token (JWT). Токен должен передаваться в HTTP-заголовке `Authorization: Bearer <token>`.

- Защита данных (Security): пароли пользователей должны хэшироваться с использованием криптографически стойких алгоритмов (например, `bcrypt`) до сохранения в базу данных. В ответах API не должны фигурировать чувствительные данные пользователей.

- Сетевая безопасность: сервер должен быть настроен на обработку CORS (Cross-Origin Resource Sharing) для обеспечения безопасных запросов со стороны веб-клиента (фронтенда).

- Обработка ошибок: API должно иметь централизованный механизм перехвата исключений, возвращающий клиенту унифицированный ответ с указанием статуса и причины ошибки.

- Документирование: API должно быть строго типизировано и описано в соответствии со спецификацией OpenAPI 3.0.

2. Выбор формата реализации API

Для реализации программного интерфейса выбран архитектурный стиль REST API.

Паттерн Backend For Frontend (BFF) целесообразно использовать в микросервисной архитектуре, когда требуется агрегация данных из разных

сервисов под специфические нужды конкретного клиента (например, мобильного приложения или смарт-ТВ). Поскольку разрабатываемое приложение представляет собой единый монолитный бэкенд, призванный отдавать универсальный набор данных, использование REST API является оптимальным. REST предоставляет стандартизированный, независимый от клиента интерфейс, базирующийся на семантике HTTP-методов (GET, POST, PATCH, DELETE).

3. Проектирование структуры эндпоинтов

На базе разработанной ER-диаграммы спроектирована структура эндпоинтов (разработанный файл спецификации `openapi.yaml` приложен к работе). Структура маршрутов (базовый путь `/api/v1`) полностью отражает сущности и связи из БД:

Метод	Эндпоинт	Описание	Доступ
Авторизация			
POST	<code>/auth/register</code>	Создание записи в таблице User	Все
POST	<code>/auth/login</code>	Аутентификация	Все
Пользователи			
GET	<code>/users/me</code>	Чтение записи в таблице User	Авторизован
PATCH	<code>/users/me</code>	Обновление записи в таблице User	Авторизован
GET	<code>/users/{id}</code>	Чтение профиля по ID	Все
POST	<code>/users/{id}/subscribe</code>	Добавление записи в Subscription	Авторизован
DELETE	<code>/users/{id}/subscribe</code>	Удаление записи в Subscription	Авторизован
Рецепты			
GET	<code>/recipes</code>	Выборка списка Recipe	Все
POST	<code>/recipes</code>	Создание записи Recipe	Авторизован
GET	<code>/recipes/{id}</code>	Чтение записи Recipe	Все
PATCH	<code>/recipes/{id}</code>	Обновление записи Recipe	Автор
DELETE	<code>/recipes/{id}</code>	Удаление записи Recipe	Автор
POST	<code>/recipes/{id}/like</code>	Добавление записи в Like	Авторизован
DELETE	<code>/recipes/{id}/like</code>	Удаление записи в Like	Авторизован
POST	<code>/recipes/{id}/save</code>	Добавление записи в SavedRecipe	Авторизован
DELETE	<code>/recipes/{id}/save</code>	Удаление записи в SavedRecipe	Авторизован
GET	<code>/recipes/{id}/comments</code>	Чтение записей в Comment	Все
POST	<code>/recipes/{id}/comments</code>	Создание записи в Comment	Авторизован

Блоги (Посты)			
GET	/blogs	Выборка списка BlogPost	Все
POST	/blogs	Создание записи BlogPost	Авторизован
GET	/blogs/{id}	Чтение записи BlogPost	Все
PATCH	/blogs/{id}	Обновление записи BlogPost	Автор
DELETE	/blogs/{id}	Удаление записи BlogPost	Автор
POST	/blogs/{id}/like	Добавление записи в Like (блог)	Авторизован
DELETE	/blogs/{id}/like	Удаление записи в Like (блог)	Авторизован
GET	/blogs/{id}/comments	Чтение записей в Comment (блог)	Все
POST	/blogs/{id}/comments	Создание записи в Comment (блог)	Авторизован
Справочники			
GET	/dish-types	Чтение словаря категорий (DishType)	Все
GET	/ingredients	Чтение словаря ингредиентов (Ingredient)	Все

4. Описание тел запросов, ответов и возможных ошибок

В рамках OpenAPI-схемы детально проработаны контракты обмена данными:

1. Тела запросов (Request Body): Для создания и обновления сущностей описаны DTO (Data Transfer Objects). Например:

- Запрос POST /recipes ожидает объект RecipeCreate с обязательными полями title (string), difficulty (string) и cooking_time_minutes (integer).

- Запрос POST /auth/register требует передачи username, email и password.

2. Тела ответов (Response Body):

- Успешные GET-запросы коллекций (например, /recipes) возвращают массив JSON-объектов Recipe.

- Запросы конкретного ресурса (например, /users/{id}) возвращают одиночный JSON-объект со структурой, соответствующей полям БД (исключая пароль).

- Успешная аутентификация возвращает объект { "access_token": "..."}.

- Операции удаления (DELETE) или снятия лайков возвращают статус 204 No Content без тела.

3. Обработка ошибок: Спроектирована унифицированная структура ошибки (схема Error), содержащая поля statusCode (integer) и message (string). Предусмотрены следующие сценарии (HTTP-статусы):

- **400 Bad Request:** ошибка валидации входящих параметров (например, неверный формат email или отсутствие обязательных полей в Request Body).

- **401 Unauthorized:** обращение к защищенным эндпоинтам (например, постановка лайка или создание поста) без передачи валидного JWT в заголовке Authorization.

- **403 Forbidden:** попытка изменения или удаления контента, не принадлежащего текущему авторизованному пользователю.

- **404 Not Found:** обращение к ресурсу с несуществующим ID (UUID не найден в БД).

Вывод

В результате выполнения домашнего задания успешно формализованы функциональные и нефункциональные требования к бэкенд-приложению. Обоснован выбор архитектурного стиля REST API. На основе спроектированной ранее реляционной базы данных был разработан перечень эндпоинтов, покрывающий всю бизнес-логику приложения. Все маршруты, форматы запросов, ответов и механизмы обработки ошибок были строго документированы в формате спецификации OpenAPI 3.0.